DEPARTMENT OF MECHANICAL ENGINEERING
RICE UNIVERSITY

MECH 498/598: INTRODUCTION TO ROBOTICS

Lab #4
Dynamic Simulation
Due Friday, May 1st 2026

Assignment Description
For this assignment, undergraduates will once again be able to work in groups of two. Only one submission (electronic only) per group is required. In this lab you will build a simple dynamic simulation of a 2DOF manipulator and then implement control of a 3DOF manipulator moving an object around in space. Some code has been provided for you on Canvas as well, with additional instructions in the comments.

Submission Instructions
Your submission for this assignment will only have an auto-graded portion (the plotting is purely for your debugging). You will turn this assignment in via GradeScope. Please put both teammates names in for joint submissions.
The submission folder should contain ALL files that you used for your development, included provided files you did not change. Several base implementation files will be provided for you to build upon in this lab for both questions.

1. In this lab, you will create software for the dynamic simulation of chain link manipulators. Begin by considering the 2-link, RR, nonplanar, manipulator shown below. **Important:** In addition to the two point masses m_1 and m_2 shown at the ends of the two links, the two links should be considered to be rigid, thin rods with masses m_{r1} and m_{r2}, respectively, which are evenly distributed throughout their volume. The dynamic equations are as follows:

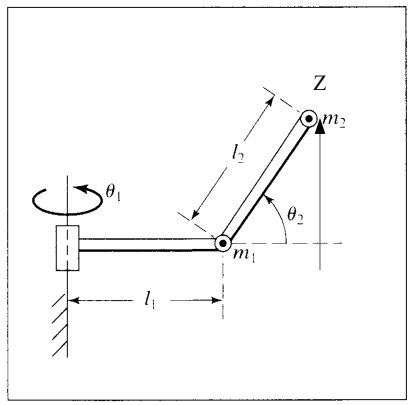
$$M_1 L_{c1}^2 \ddot{\theta}_1 + M_2 (L_1 + L_{c2}\cos\theta_2)^2 \ddot{\theta}_1 + (I_1 + I_2)\ddot{\theta}_1 - 2M_2 L_{c2}\sin\theta_2\,(L_1 + L_{c2}\cos\theta_2)\dot{\theta}_1\dot{\theta}_2 = \tau_1$$

$$M_2 L_{c2}^2 \ddot{\theta}_2 + I_2\ddot{\theta}_2 + M_2 L_{c2}\sin\theta_2\,(L_1 + L_{c2}\cos\theta_2)\dot{\theta}_1^2 + M_2 g L_{c2}\cos\theta_2 = \tau_2$$

$$M_1 = m_1 + m_{r1} \qquad M_2 = m_2 + m_{r2}$$

$$L_{c1} = \frac{\left(m_1 + \frac{1}{2}m_{r1}\right)L_1}{M_1} \qquad L_{c2} = \frac{\left(m_2 + \frac{1}{2}m_{r2}\right)L_2}{M_2}$$

$$I_1 = \frac{1}{12}m_{r1}L_1^2 + m_1\left(\frac{L_1}{2}\right)^2 \qquad I_2 = \frac{1}{12}m_{r2}L_2^2 + m_2\left(\frac{L_2}{2}\right)^2$$



Use the following constants: $m_1 = 1, m_2 = 5, m_{r1} = 2.3, m_{r2} = 2.3, l_1 = 1, l_2 = 1.41,$ (defined in RRBot.py) with the manipulator initially at rest in the position $\theta_1 = \frac{\pi}{3}$ and $\theta_2 = \frac{\pi}{2}$.

A simple approach to building a dynamic simulation consists of the following steps:

- Specify and store the initial state of the system. The state includes θ and $\dot{\theta}$ for each joint.
- Next, repeat the following three steps. During repetition, time is incremented from 0 until some final time t_f in small increments dt.
  - Sum the torques on each link, τ, at the current instant in time. If there are no torques (that is, there is nothing applied externally and no control feedback), $\tau = 0$.
  - To compute the acceleration of each joint, $\ddot{\theta}$, start with the dynamic model of the system given in class: $\tau = M(\boldsymbol{\theta})\ddot{\boldsymbol{\theta}} + C(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}}) + G(\boldsymbol{\theta})$. Next, solve for the vector $\ddot{\boldsymbol{\theta}}$ using the inverse of the inertia matrix $M(\boldsymbol{\theta})$.
  - Calculate the new state of the system by numerical integration of the acceleration for each joint. A simple numerical integration technique is the trapezoidal rule: at time step i, the velocity is $v_i = v_{i-1} + 0.5(a_{i-1} + a_i)dt$, and the position is $p_i = p_{i-1} + 0.5(v_{i-1} + v_i)dt$. (This formula implies that the updated derivative a_i or v_i has been calculated using the states from the previous iteration i-1.) This function has been created and provided for you in RRBot.py

The sample Python file RRBot.py is provided to you on Canvas. The RRBot.py defines the robot, sets up the joints, provides the forward kinematics, the inverse kinematics, and the drawing capabilities. Your job is to create a simulation using these provided capabilities. [30 pts]

a. Modify RRBot.py given to create a dynamic simulation of the manipulator above. The simulation should last 10 seconds, with increments of 0.01 seconds. Play with the values of the physical parameters in the system, as well as the initial conditions. Watch the effects in the video output of your simulation.

b. A good way to check that your simulation is working properly is to plot the total energy of the system over time. Find the kinetic and potential energy of the RR robot and generate the plot within the loop of your simulation code.

Note: The code provides plotting functions provided that you properly fill in the intermediate information during the simulation (KE_data, PE_data, E_total_data, joint_1_data, joint_2_data, tau_data). Use these to visualize and troubleshoot.

Note: The total energy will not be exactly 0, this is due to Python rounding. If it moves up or down a *little* bit that is expected.

2. Now you will add a simple controller to your simulation. Use a proportional-derivative feedback controller for each joint, so that $\tau = -K_p(\theta - \theta_d) - K_v\dot{\theta}$. To more closely represent a physical system, your joint torques should be limited to $-200 < \tau < 200$. Torques outside this limit should be limited to the max or min value.
   - Part 1: Set Point 1 Underdamped – attempt to reach the desired location of $\theta_{1d} = -\pi$ and $\theta_{2d} = -\pi/3$ with an underdamped response.
   - Part 2: Set Point 1 Critically Damped – attempt to reach the desired location of $\theta_{1d} = -\pi$ and $\theta_{2d} = -\pi/3$ with a critically damped response.
   - Part 3: Set Point 2 Critically Damped – attempt to reach a desired location of $\theta_{1d} = -\frac{\pi}{2}$ and $\theta_{2d} = 1.3$ with a critically damped response using the same gains as Set Point 1.

   Note: The autograder will call the functions **`test_setpoint_X_`** to validate the behavior of the systems
   Note: If you need to define different gains for each joint, that is fine.

3. Now it's time to meet Rob the 3-link robot. Rob looks an awful lot like the first three links of the FANUC robot you encountered in Lab 2. The Python code you need to define and simulate this robot is provided to you via a c++ binary. [50 points]

   Rob has a single task to accomplish – he needs to pick up a heavy magnetic ball, move it to a new location, and then return to his home configuration. Fortunately, Rob's end effector is equipped with a powerful electromagnet that can be flipped on and off, so picking up and placing the ball is as simple as having the end effector within a certain distance of the target position while the end effector velocity is very near 0. You need to design the robot trajectory and controller to move Rob from its starting position, to the starting position of the ball where you come to a near stop, move to the final position of the ball, again coming to a complete stop, and finally to the final position and coming to a stop. If Rob is not moving slowly enough, or not near enough to the ball, it will not pick up or drop it.

   `RobBase.py` – This file defines the robot, sets up the joints, sets up the drawing capabilities, calculates the forward and inverse kinematics, and a few other things.

   `RobSimulation.py` – This class is *derived* from the **`RobBase.py`** class. This class implements the simulation via a Python wrapped C++ implementation. (This is so you can't cheat on how to make the simulation for question 1 or back-calculate the dynamics so you have to control it without the model!). Inside of **`RobSimulation.py`**, the full dynamics of the robot are calculated, and a simulation is done.

   `RobStudent.py` -- Your job is to fill out the file **`RobStudent.py`**. **`RobStudent.py`** is *derived* from **`RobSimulation.py`**, which is derived from **`RobBase.py`**. This means that all of the functions in RobBase and RobSimulation are available to you in **`RobStudent.py`**

   You must fill out the two functions provided, in **`RobStudent.py`** and you can use the provided test script to run the simulation, **`student_rob_test.py`**

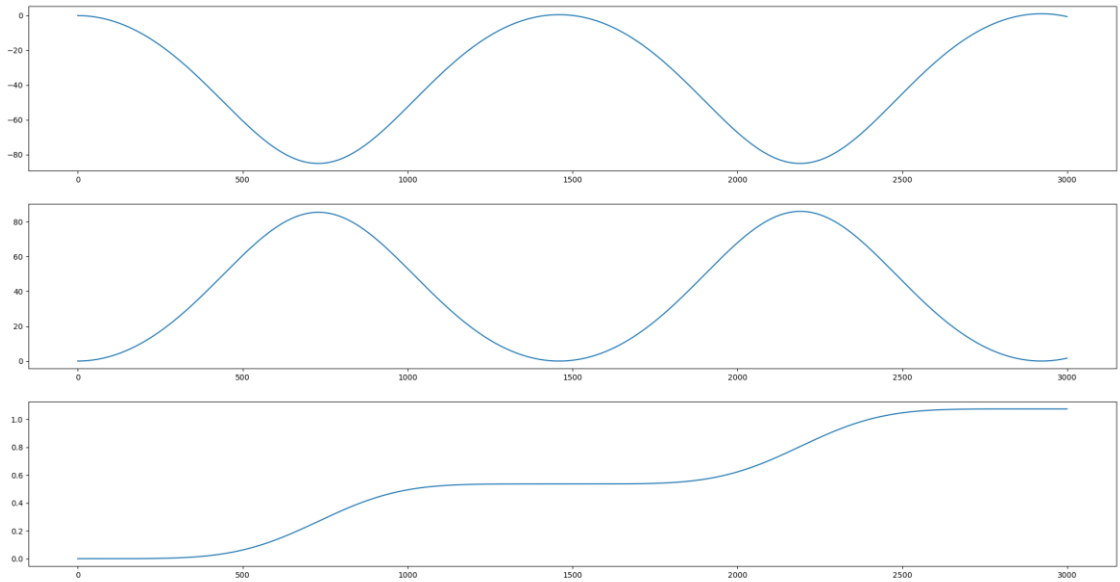
   `createRobTrajectory(waypoints)` – given the four waypoitns, you need to generate a trajectory that includes the timesteps, joint angular positions, and joint angular velocities that you want to achieve throughout

the motion. You first need to interpolate the cartesian position from point to point (like lab 3) then using the provided calculate_ik function, calculate the desired joint positions to achieve this trajectory, and finally the desired joint velocities to complete this trajectory.
Hint: It might be helpful to have a small dwell time at each goal position.

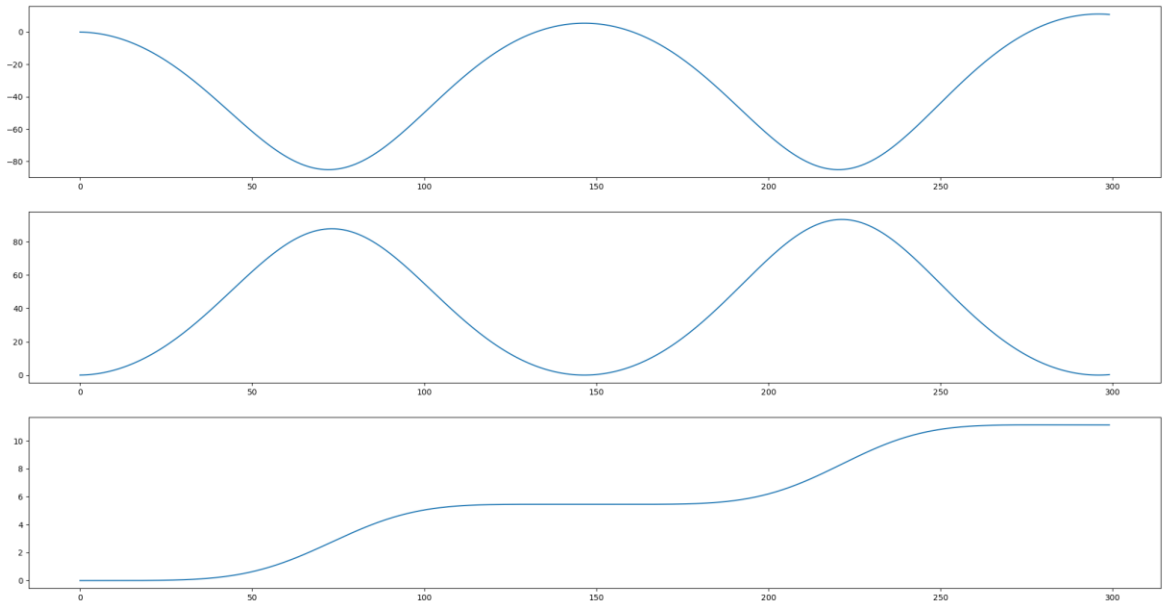
`get_rob_torque(theta, theta_dot, timestep)` – this is where the magic happens. Given the robots current angles (theta), current velocities (theta_dot), and current timestep you must first find the desired position and velocity at that timestep (using the trajectory you created), calculate the error in position and velocity, and calculate the desired joint torques to control the robot. You should use a PD controller with gravity compensation to calculate your desired torques (tau). Try completing the task without gravity compensation first, and you will notice why it's necessary.

Notes and Tips:

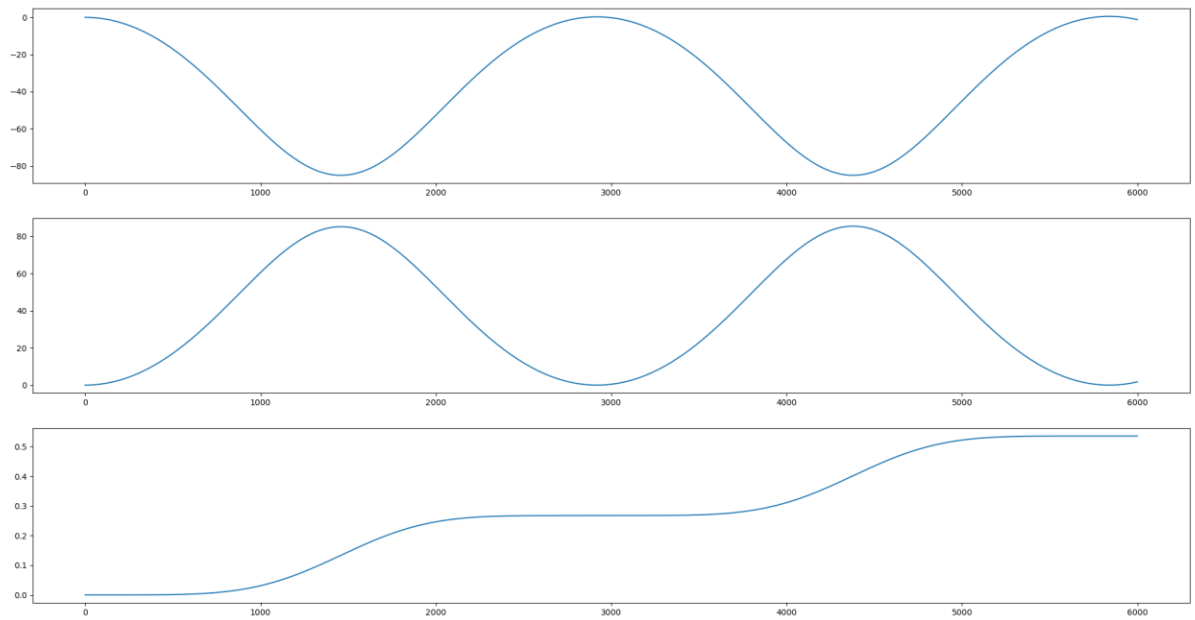
- On your first iteration, use the initial values of theta1, theta2, theta1d, theta2d, and your computed PD control inputs, tau1 and tau2, to solve theta1dd and theta2dd. On subsequent iterations, use the previous values of theta1, theta2, theta1d, and theta2d, and a new computation of the PD control inputs to solve for theta1dd and theta2dd. Integrate theta1dd and theta2dd twice to get velocity and positions. Use these on the next iteration.


0
−20
−40
−60
−80
0
500
1000
1500
2000
2500
3000
80
60
40
20
0
0
500
1000
1500
2000
2500
3000
1.0
0.8
0.6
0.4
0.2
0.0
0
500
1000
1500
2000
2500
3000

0.001


0
−20
−40
−60
−80
0
50
100
150
200
250
300
80
60
40
20
0
0
50
100
150
200
250
300
10
8
6
4
2
0
0
50
100
150
200
250
300

0.01


0
−20
−40
−60
−80
0
1000
2000
3000
4000
5000
6000
80
60
40
20
0
0
1000
2000
3000
4000
5000
6000
0.5
0.4
0.3
0.2
0.1
0.0
0
1000
2000
3000
4000
5000
6000

0.0005